

```

/*****
// TD : fonctions de paramètres :
// Passage par nom et/ou par adresse des fcts
*****/
#include <stdio.h>

int somme(int a, int b){
    return(a+b);
}

int diff(int a, int b){
    return(a-b);
}

int produit(int a, int b){
    return (a*b);
}

// b !=0
int division(int a, int b){
    return (a/b);
}

int modulo(int a, int b){
    return (a%b);
}

// fct en Parametre
int OperatBinaire(int x, int y, int (*f)(int, int) ){
    //appel de f sur les arguments x et y
    return ((*f)(x,y) );
}

void main(void){
    printf("\n Operations Passage par nom fcts");
    printf("\n %d ",OperatBinaire(3, 4, somme) );
    printf("\n %d ",OperatBinaire(3, 4, diff) );
    printf("\n %d ",OperatBinaire(3, 4, produit) );
    printf("\n %d ",OperatBinaire(3, 4, division) );
    printf("\n %d ",OperatBinaire(3, 4, modulo) );

    printf("\n Operations Passage par adresse des fcts");
    printf("\n %d ",OperatBinaire(3, 4, &somme) );
    printf("\n %d ",OperatBinaire(3, 4, &diff) );
    printf("\n %d ",OperatBinaire(3, 4, &produit) );
    printf("\n %d ",OperatBinaire(3, 4, &division) );
    printf("\n %d ",OperatBinaire(3, 4, &modulo) );

    printf("\n");
}

```

```

/*****
// TD : fonctions de parametres :
// Passage par nom et/ou par adresse des fcts
*****/
#include <stdio.h>

//Renommage fct en parametre = Def Type Fct
typedef int (*Fptr)(int, int);

// Passage d une fct en parametre a une fct
int OperatBinaire(int x, int y, Fptr f ){
    //appel de f sur les arguments x et y
    return ((*f)(x,y) );
}

int somme(int a, int b){
    return(a+b);
}

int diff(int a, int b){
    return(a-b);
}

int produit(int a, int b){
    return (a*b);
}

// b !=0
int division(int a, int b){
    return (a/b);
}

int modulo(int a, int b){
    return (a%b);
}

void main(void){
    printf("\n Operations passage par adresse des fcts");
    printf("\n %d ",OperatBinaire(3, 4, &somme) );
    printf("\n %d ",OperatBinaire(3, 4, &diff) );
    printf("\n %d ",OperatBinaire(3, 4, &produit) );
    printf("\n %d ",OperatBinaire(3, 4, &division) );
    printf("\n %d ",OperatBinaire(3, 4, &modulo) );

    printf("\n Operations passage par nom des fct");
    printf("\n %d ",OperatBinaire(3, 4, somme) );
    printf("\n %d ",OperatBinaire(3, 4, diff) );
    printf("\n %d ",OperatBinaire(3, 4, produit) );
    printf("\n %d ",OperatBinaire(3, 4, division) );
    printf("\n %d ",OperatBinaire(3, 4, modulo) );

    printf("\n");
}

```



```

/*****/
// TD : Tableau de fonctions
// Passage par nom et/ou par adresse des Fcts
/*****/
#include <stdio.h>
#include <stdlib.h>
#define DIM 5
// Def Type Fct
typedef int (*Fptr)(int, int);

// Def Type Tableau Statique de Fct
typedef Fptr TabFctS[DIM];

// fct en Parametre
int OperatBinaire(int x, int y, Fptr f){
    //appel de f sur les arguments x et y
    return ((*f)(x,y) );
}

// Allocation Memoire d un tableau Fonction
Fptr *allocTabFct(int nMax){
Fptr *tFct;
    tFct=(Fptr *)malloc(sizeof(Fptr)*nMax);
    if(tFct==NULL){
        printf("\n Pb alloc Mem");
        exit(1);
    }
    return tFct;
}

// Liberation Memoire d un tableau Fonction
Fptr *libTabFct(Fptr *tFct){
    free(tFct);
    tFct=NULL;
    return tFct;
}

int somme(int a, int b){
    return(a+b);
}
int diff(int a, int b){
    return(a-b);
}
int produit(int a, int b){
    return (a*b);
}
// b !=0
int division(int a, int b){
    return (a/b);
}

int modulo(int a, int b){
    return (a%b);
}

```

```

}

void main(void){
// tFD : var de type Tableau Dynamique de fct
Fptr *tFD;
// tFS : var de type Tableau Statique de fct
TabFctS tFS1={somme, diff, produit, division, modulo};
TabFctS tFS2={&somme, &diff, &produit, &division, &modulo};
int i, nbFct=5;

    // Alloc Mem Tab Fct
    tFD=allocTabFct(nbFct);

    // Init = Passage par adresse
    tFD[0]=&somme; tFD[1]=&diff; tFD[2]=&produit;
    tFD[3]=&division; tFD[4]=&modulo;
    printf("\n Operations passage par nom");
    for(i=0; i<nbFct; i++){
        printf("\n tFS[%d]=%d ",i, OperatBinaire(15, 4, tFS1[i]));
        printf("tFD[%d]=%d ",i, OperatBinaire(15, 4, tFD[i]) );
    }
    printf("\n");

    // Init = Passage par nom
    tFD[0]=somme; tFD[1]=diff; tFD[2]=produit;
    tFD[3]=division; tFD[4]=modulo;
    printf("\n Operations passage par adresse");
    for(i=0; i<nbFct; i++){
        printf("\n tFS[%d]=%d ",i, OperatBinaire(15, 4, tFS2[i]));
        printf("tFD[%d]=%d ",i, OperatBinaire(15, 4, tFD[i]) );
    }
    printf("\n");

    // Lib Mem Tab Fct
    tFD = libTabFct(tFD);
}

```

```

/*****/
// TD : fonctions de parametres :
// Passage par nom et/ou par adresse des fcts
// Recherche avec sentinelle d'un elt ds un tableau
/*****/

#include <stdio.h>

#define DIM 10

// def TElement entier
typedef int TElement;

// def Type Fct a deux parametres
typedef int (*FComp)(TElement, TElement);

// fct egalite entre deux entiers
int estegaliteInt(TElement i, TElement j){
    return i==j;
}

/*****/
// Recherche d'un element ds un tableau compose de n elements
// Methode sequentielle avec sentinelle
/*****/
int estRecherche(int n,TElement* tab,TElement elt, FComp estEgalite) {
int i;
    //Ajout d'une sentinelle
    tab[n]=elt;
    //Parcours sequentiel avec arr^at
    i=0;
    while(! ((*estEgalite)(elt,tab[i])) )
        i++;
    return i<n;
}

void main(void){
    TElement tab[DIM]={5, 0, 4, 1, 7}, elt=7;
    int n=5;

    printf("\n EstRecher=%d \n ", estRecherche(n, tab, elt, estEgaliteInt));
}

```

```

/*****/
// TD : fonctions de parametres :
// Passage par nom et/ou par adresse des fcts
// Recherche d un elt ds un tableau
/*****/
#include <stdio.h>
#define DIM 10

// Def Type Complexe
typedef struct{
    double pr;
    double pi;
} TElement;

// def Type Fct a deux parametres
typedef int (*FComp)(TElement, TElement);

// Primitives du type Complexe
double pReel(TElement c){
    return c.pr;
}
double pIm(TElement c){
    return c.pi;
}
// fct egalite entre deux complexes
int estEgaliteComp(TElement c1, TElement c2){
    return pReel(c1)==pReel(c2) && pIm(c1)==pIm(c2);
}
/*****/
// Recherche d'un élément ds un tableau composé de n éléments
// Methode sequentielle avec sentinelle
/*****/
int estRecherche(int n, TElement* tab, TElement elt, FComp estEgalite) {
    int i;
    //Ajout d'une sentinelle
    tab[n]=elt;
    //Parcours séquentiel avec arrêt
    i=0;
    while(! ((*estEgalite)(elt,tab[i])) )
        i++;
    return i<n;
}

void main(void){
    TElement tab[DIM]={5, 5}, {0, 0}, {4, 4}, {1, 1}, {7,7}}, elt={7,7};
    int n=5;
    // Passage par nom
    printf("\n Passage Fct par Nom");
    printf("\n estRecher=%d \n ", estRecherche(n, tab, elt, estEgaliteComp));
    // Passage par adresse
    printf("\n Passage Fct par adresse");
    printf("\n estRecher=%d \n ", estRecherche(n, tab, elt, &estEgaliteComp));
}

```